

# Map Integration

# Kursüberblick

1. Intro
2. Mobile UX & Input
3. Performance & Mobile Constraints
4. GPS & Location-Based Mechanics
-  5. Map Integration
6. AR Foundation
7. Game Loop & Systems
8. Polishing + Smartphone Features
9. Prüfung

# Heute

- Weltkarten und Map SDK's
- Map SDK Integration (Cesium & MapTiler)
- Custom Map erstellen
- World-Space vs Map-Space
- Marker setzen
- Spielerposition visualisieren
- Creature Spawnen

# Weltkarten

# Weltkarte - Status Quo

Wo stehen wir bisher?

Wo

Lat 2012312  
Long 123123  
Alt 123123  
Mode:

Swap Mode

Pos 1  
Pos 2  
Pos 3

Stroll North  
Stroll East

Catch!

Target  
Lat: 49.41399  
Long: 8.65110  
Distance: 500m

Pos 1 49.41399 / 8.65110  
Pos 2 49.41423 / 8.65092  
Pos 3 49.38160 / 8.67010



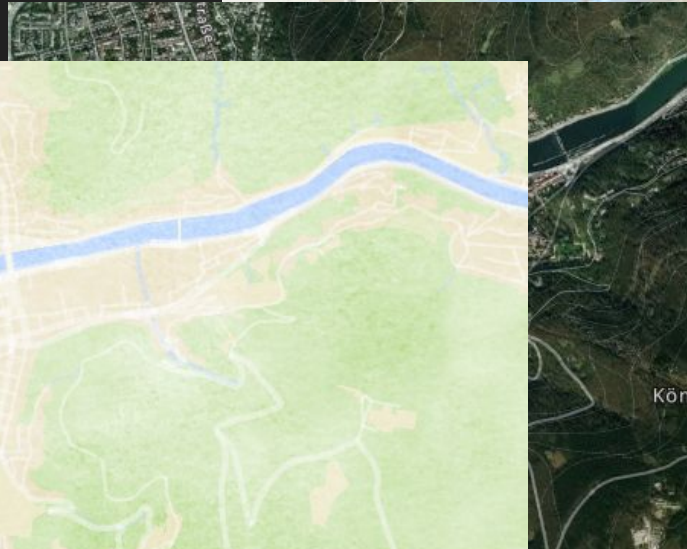
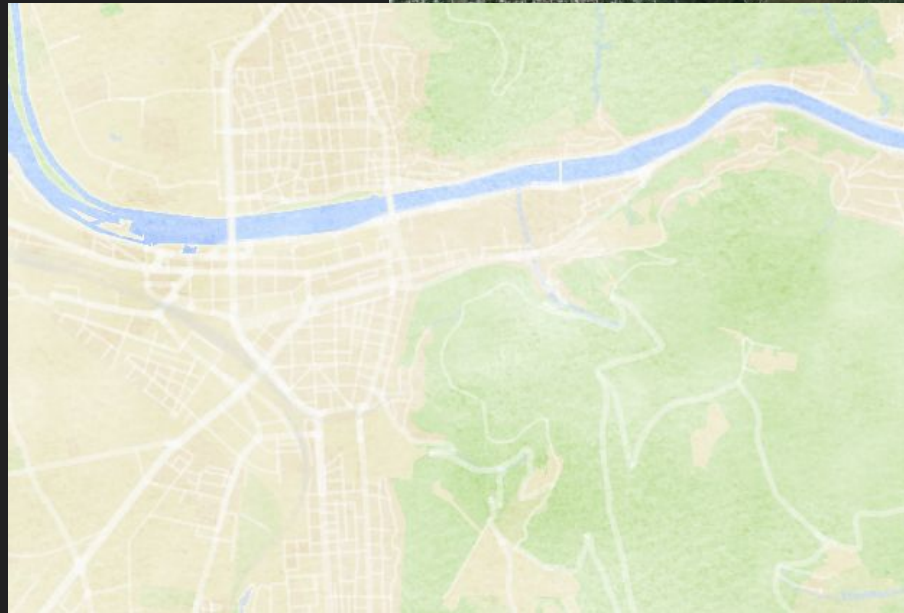
# Weltkarte

Was ist eine “digitale Weltkarte”?

Wozu brauchen wir sie?

Was macht sie aus?

# Kartenarten



# Rolle eines Map SDK

Karten sind Projektion einer Kugel auf eine Fläche

- Dabei entstehen Verzerrungen -> für uns zwar vernachlässigbar
- Map Tiling
  - schnelleres Laden und zoomen
  - werden dynamisch geladen, je nach Zoom Level
- Rendering der Map-Tiles
- Layering
  - Straßen
  - Orte
  - Namen



# Map SDK Integration

# Map SDK's

- Cesium
- Maptiler
- Mapbox
- MapLibre
- OSM (Open Street Maps)

# Setup Maptiler und Cesium

1. <https://cloud.maptiler.com/maps/aquarelle-v4/> öffnen
  - a. Account anlegen
2. <https://docs.maptiler.com/unity/> - Anleitung befolgen - bis "Add global real-world 3D terrain to your level"

**Dort stoppen! Das brauchen wir nicht mehr!**

# Cesium Testen

1. MainCamera -> Ortho, Pos y=50, Rot x=60

2. Cesium Fenster öffnen

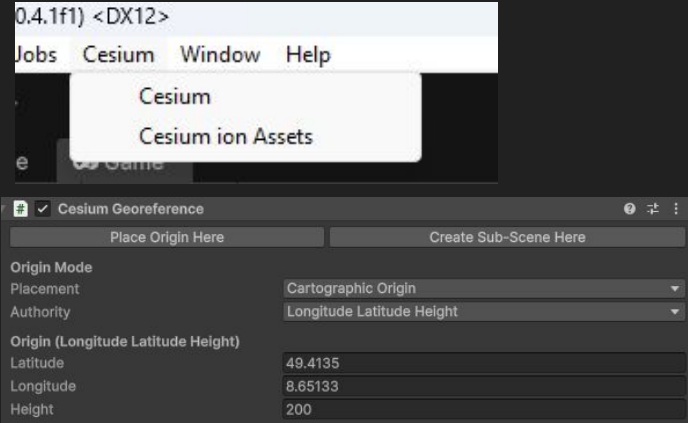
Connect Cesium anklicken

3. Google Photorealistic 3D Tiles hinzufügen

4. Cesium Georeference anklicken

4.a Origin ändern auf SRH (49.4135, 8.65133, 200)

6. Kamera bewegen und staunen!



# Cesium mit Maptile verknüpfen

1. Auf Maptiler Map raussuchen
  - a. <https://cloud.maptiler.com/maps/aquarelle-v4/>
  - b. Kapitel "Raster Tiles" -> 512er Map, 2. Link (mit {z},{x},{y}) kopieren
2. Google Photorealistic 3D Tiles wieder löschen
3. Cesium Fenster öffnen
  - a. Cesium World Terrain + Bing Maps Aerial hinzufügen
4. Cesium World Terrain anklicken
  - a. Cesium Ion Raster Overlay Komponente löschen -> weiße map
  - b. Neue Komponente: "CesiumURLTemplateRasterOverlay"
  - c. unsere Map-URL (1.b) von Maptiler bei TemplateURL einfügen
  - d. in der URL {y} zu {reverseY} ändern

# Finetuning

1. In `CesiumURLTemplateRasterOverlay`
  - a. Tile Width und Tile Height jeweils auf 512 stellen.
  - b. Maximum Level auf 50
2. Camera neu einstellen
  - a. Position (0, 200, 0)
  - b. Rotation (90, 0, 0)
  - c. Orthographic Size (40)

# Troubleshoot

1. [error] [ErrorList.h:89] Errors while loading image for tile:
  - Received response code 429 for image

Code 429 = zu viele Requests -> Timeout

Im **3D Tileset**: **Update in Editor** deaktivieren

-> warten bis Timeout ausgelaufen oder Build machen

# Custom Map



# Custom Map in Cesium

1. Im Free-Tier leider nicht nutzbar, ein Kurzer Blick lohnt aber trotzdem  
-> Mapbox hat ein ähnliches Tool
2. Auf Maptiler Map raussuchen
  - a. <https://cloud.maptiler.com/maps/aquarelle-v4/>
3. Customize a Copy
4. Map customizen
  - a. z.B. im Layer Built-Up -> Visualization auf Extrusion
  - b. Path -> Pattern auf green
  - c. Neuen Layer hinzufügen -> Satellite, Opacity 20%
5. Map benennen
6. Map publishen

# Alternativen

1. Maputnik <https://maplibre.org/maputnik/>
2. Stadia
3. Mapbox (Free-Tier, aber Kreditkarte hinterlegen) => am besten für Unity

Kurze Pause?

Spieler einbinden

# Optimieren des Kartenausschnitts

Kamera anpassen:

- Position: (0,200,0)
- Rotation (90,0,0)
- Size 100

# Map-Follow-Typen

## 1. Map Centric

- a. Karte ist fest. Spieler bewegt sich über die Map
- b. "normale Game-Level"
- c. bei riesiger Karte -> Große Positionswerte -> float Ungenauigkeit & Jitter

## 2. Player Centric

- a. Spieler ist fest, Karte bewegt sich unter dem Spieler durch.
- b. Z.B. bei den originalen Pokemon Games (Blau & Rot)
- c. Tiles müssen dynamisch geladen und eingefügt werden

# Karte mit Position verknüpfen

1. **CesiumGeoreference** und **MainCamera** zu Prefabs machen
- 2.
3. Alte LocationSzene kopieren und umbenennen -> Map Scene
  - a. Map scene öffnen
  - b. testen ob die Location noch funktioniert
4. **CesiumGeoreference** und **MainCamera** Prefabs einfügen
  - a. (beide auf Position 0/0/0 setzen
5. LocationScript von letzter Vorlesung öffnen
  - a. erweitern, kopieren, oder neuschreiben!

# LocationScript mit Georeference Verknüpfen

1. Neues Feld: `[SerializeField] CesiumGeoreference _cesiumGeoRef;`

2. In der Location Update Funktion:

a. Sowohl für die “echte” Location, als auch für die Mock:

```
_cesiumGeoRef.SetOriginLongitudeLatitudeHeight(_playerCoords.y, _playerCoords.x,  
_playerCoordsAlt);
```

b. **Format beachten: Long (!), Lat, Alt**

3. CesiumGeoreference im Inspector setzen

4. Testen

5. Wenns nicht funktioniert, schreien: **“Du Scheissscript”**



# Dynamische Kamerahöhe

Aktuell sitzt die Kamera fest 200 Einheiten über der Karte.

Aber wir haben ein dynamisches Terrain-Mesh. Wenn wir in den Bergen oder am Meer sind, Gefahr von **Near/Far-Clipping** der Kamera

Wir müssen die Kamera dynamisch verschieben.

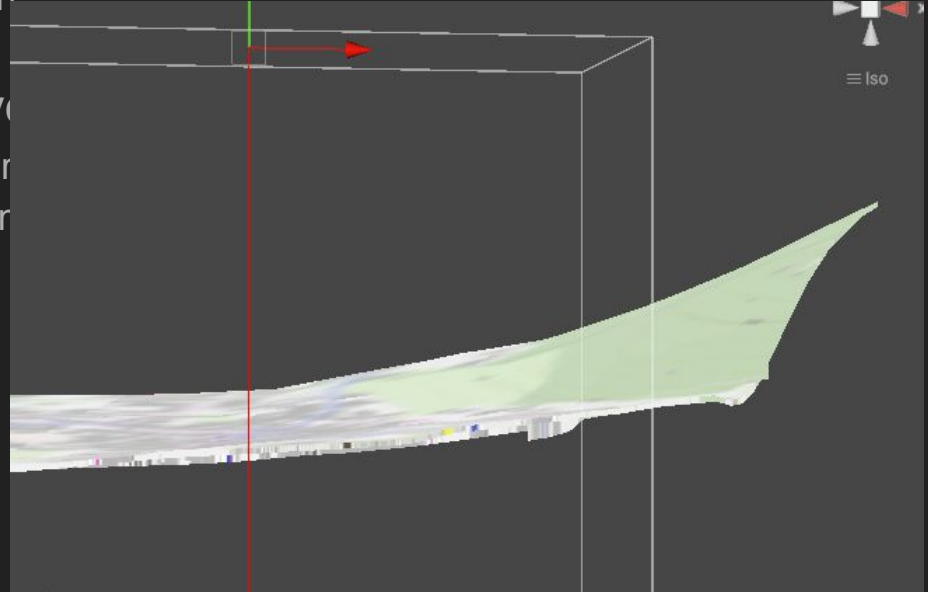
Aber wie?

# Dynamische Kamerahöhe - Optionen

1. Direkt an die Höhendaten aus der Location knüpfen
  - a. Schlechte Idee. Höhendaten sind sehr ungenau
  - b. Dadurch würde unsere Kamera springen
2. Kamera immer relativ zum Terrain v
  - a. gute Idee. Terrain haben wir. Aber woher
  - b. Einfachste Lösung: Raycast von der Kar

Wenn der Raycast das Terrain trifft:

=>  $\text{KameraPosition} = \text{Hitpoint} + 200y$



# Dynamische Kamerahöhe - Raycast

Mögliches weiteres Problem:

Was wenn wir mit hohem Terrain starten? Oder sehr steile Klippen haben?

- Plötzlich befindet sich die Kamera unter dem Terrain und der Raycast trifft nicht!

“Better safe than sorry!”

-> Starten des Raycast von **über der Kamera!**

# Dynamische Kamerahöhe - Script

Script auf der Kamera erstellen -> **CameraHeightAdjuster**

```
[SerializeField] float _cameraOffset = 200;

public void AdjustHeight(){

    Ray ray = new Ray(transform.position + Vector3.up * 2000, Vector3.down);

    if (Physics.Raycast(ray, out RaycastHit hit)) {

        Vector3 pos = transform.position;

        pos.y = hit.point.y + _cameraOffset;

        transform.position = pos; }}}
```

**AdjustHeight()** bei jedem Update der Karte aufrufen.

# Spieler visualisieren

## 1. Spieler Asset importieren

z.B. <https://github.com/Tillomaticus/MobileGameProg26Assets/blob/main/Kenney%20-%20Mini%20Characters.unitypackage>

Asset braucht mindestens Walk und Idle Animation!

## 2. Spieler-Asset auf Origin setzen Position (0/0/0) setzen

## 3. Spieler Asset muss nach mit Rotation (0/0/0) nach Norden (Z+) schauen

- a. Falls nicht, ein neues Parent erstellen, das Rotation (0/0/0) hat und dann das Child so drehen, dass es nach Norden schaut
- b. Wichtig für später!

# Spieler drehen

Der Spieler Avatar soll jetzt in die richtige Richtung schauen.

- Unsere Karte ist Nord-zentriert
- Unsere Szene ist Nord-zentriert
- Heading auf dem Kompass ist 0-360° Grad
- Rotation in Unity ist 0-360° Grad

Müssen also “nur” die Kompass Richtung abfragen und den Spieler in die Richtung drehen

# Spieler drehen

Script “**PlayerAvatar**” erstellen

## Variable

```
[SerializeField] float _rotationSpeed = 5f;
```

## Funktion

```
public void SetLookDirection(float heading){  
    Quaternion targetRotation = Quaternion.Euler(0, heading, 0);  
    transform.rotation = Quaternion.Lerp(  
        transform.rotation,  
        targetRotation,  
        Time.deltaTime * _rotationSpeed);  
}
```

# Kompass

Kompass in Unity: `Input.compass`

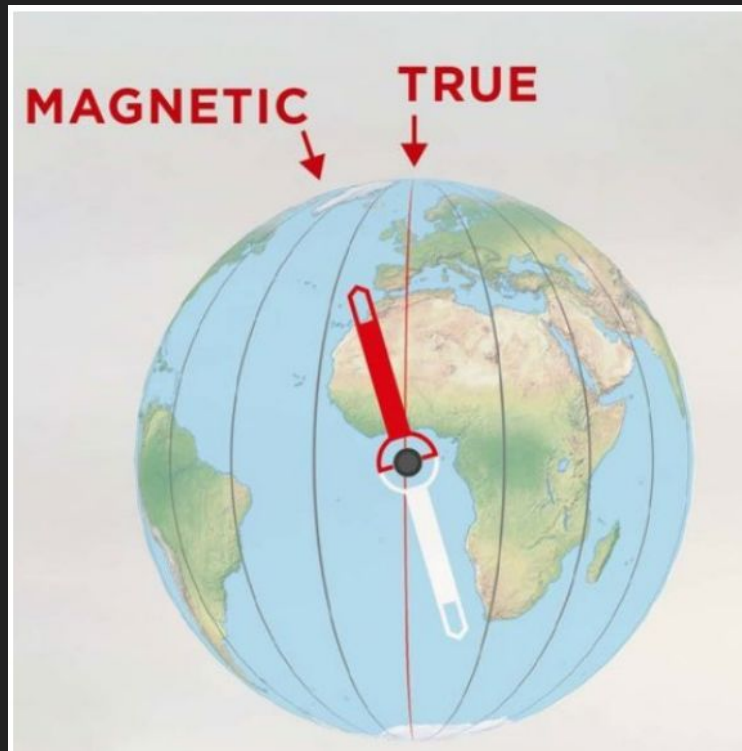
- auf Android ohne Permission abrufbar!

## MagneticHeading

- zeigt zum **magnetischen** Nordpol

## TrueHeading

- zeigt zum **geographischen** Nordpol





# Unity nervt

Der Kompass `Input.compass` ist ein API-Feature des alten Legacy Input Systems

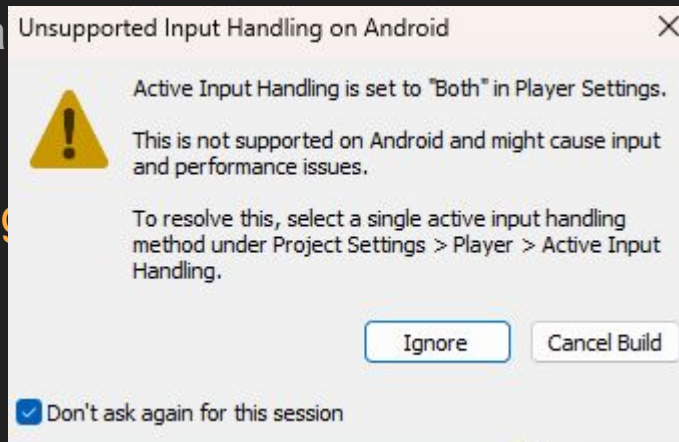
Mit dem neuen Input System liefert das immer 0.

Es gibt aber noch kein vergleichbares Feature für das neue Input System...

Deswegen müssen wir beide Input Systeme pa

PlayerSettings -> Other -> Active Input Handling

Warnung beim Building einfach ignorieren



# Kompass implementieren

In **PlayerLocation** Script

1. Kompass in **StartLocationService** anschalten `Input.compass.enabled = true`

2. Kompassdaten während des LocationUpdates abfragen

```
heading = Input.compass.trueHeading; // 0° = Nord, 90° = Ost
```

3. (empfohlen) MockDaten hinzufügen

```
[SerializeField] float _mockHeading = 0;
```

```
if (_useMockLocation)
```

```
    heading = _mockHeading;
```

4. **PlayerAvatar** im Script hinzufügen

5. nach dem Abfragen der Kompassdaten, diese dem PlayerAvatar übergeben

```
_playerAvatar.SetLookDirection(heading);
```

# Testen

- Verknüpfen der Scripts im Inspector nicht vergessen!

Funktioniert irgendwie nicht richtig, oder? Warum?

Drehung des Spielers hängt an LocationUpdate!

Umbauen:

- SetLookDirection setzt nur Ziel-Heading
- eigentliche Rotationsberechnung in über Update() aufrufen

# Umbaulösung

```
float _currentHeading=0;  
float _targetHeading;
```

```
public void SetLookDirection(float heading){  
    _targetHeading = heading;}  
}
```

```
void Update () {  
    if (_currentHeading == _targetHeading)  
        return;  
    RotateTowardsTarget ();  
}
```

```
void RotateTowardsTarget () {  
    Quaternion targetRotation = Quaternion.Euler(0, _targetHeading, 0);  
    transform.rotation = Quaternion.Lerp(  
        transform.rotation,  
        targetRotation,  
        Time.deltaTime * _roationSpeed);  
}
```

```
_currentHeading = this.transform.rotation.y;  
}
```

# Erneut Testen

Na geht doch!

Trotzdem noch ein Problem: Character muss **auf der Map** stehen!

Also denselben Raycast wie auf der Kamera auch für den PlayerAvatar nutzen!

Nur **ohne Offset!**

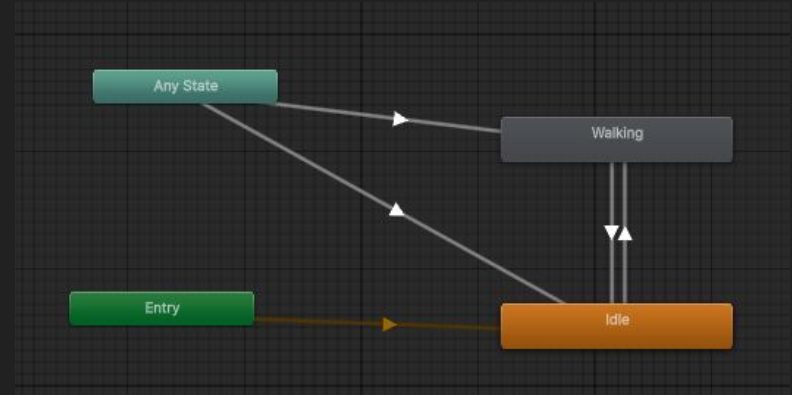
Auch an gleicher Stelle in PlayerLocation aufrufen.

Spieler animieren

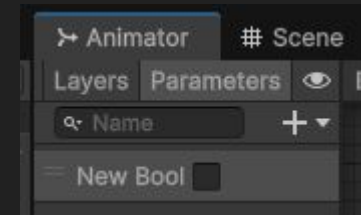
# Spieler animieren

Brauchen zwei Animationen:

1. Idle
2. Walking



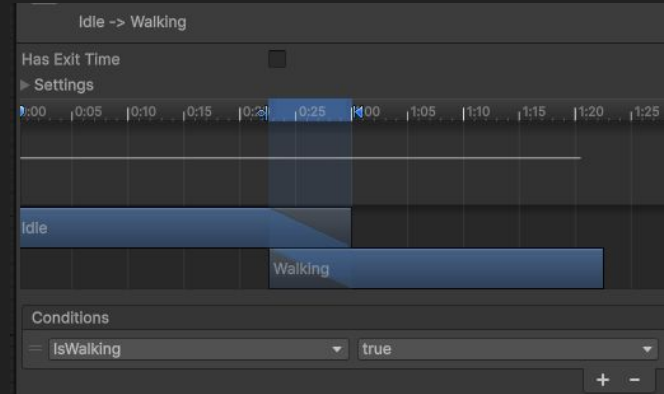
1. Create -> Animation -> Animator Controller,
2. Im AnimContr: 2x Create State -> Empty
3. Animationen in States ablegen. Idle -> als **Default (Orange)**
4. Transitions anlegen (siehe Bild)
5. Neuen Parameter erstellen: **bool IsWalking**



# Spieler animieren II

## Transitions konfigurieren:

1. AnyState > Walking/Idle:
  - a. **HasExitTime**: true
2. Walking > Idle / Idle > Walking
  - a. **HasExitTime**: false
3. In den Transitions, die Condition einstellen:
  - Für AnyState -> **Idle**: **IsWalking** false
  - Für AnyState -> **Walking**: **IsWalking** true
  - Für Walking -> **Idle**: **IsWalking** false
  - Für Idle -> **Walking**: **IsWalking** true





# Spieler Animation testen

1. **Animator** auf Spieler Avatar ziehen -> **Animation Controller** verknüpfen
2. Play drücken
  - a. Idle sollte spielen und loopen
3. **AnimatorWindow** öffnen (am besten parallel zum GameView)
  - a. **IsWalking** anschalten (Box anklicken)
  - b. die Transition zu Walking startet
  - c. Walking loopt
  - d. **IsWalking** ausschalten
  - e. die Transition zu Idle startet
  - f. Idle loopt
4. Falls nicht, schreien "**Du Scheissanimation**"
5. (optional) Im State den Animationsspeed anpassen

# Spieler-Animation per Code steuern

Script “**PlayerAvatar**” öffnen und erweitern

## Variablen

```
[SerializeField] Animator animator;  
public enum PlayerState { Idle, Walking}  
PlayerState _playerState = PlayerState.Idle;
```

## Funktionen

```
public void SetPlayerState(PlayerState newPlayerState) {  
    playerState = newPlayerState;  
    SetAnimation(); }  

```

```
void SetAnimation() {  
    if (_playerState == PlayerState.Walking)  
        _animator.SetBool("IsWalking", true);  
    else  
        _animator.SetBool("IsWalking", true);  
}
```

# Animation an Bewegung knüpfen

## Mehrere Optionen

### 1. An GPS knüpfen

- a. evtl. ungenau
- b. GPS aktualisiert nicht oft genug

### 2. Accelerometer

- a. reaktiv und ständig Updates
- b. aber schon bei jeder kleinsten Bewegung

### 3. Kombination aus beiden

- a. z.B.: Wenn GPS gerade nichts liefert, aber Accelerometer noch Bewegung erkennt -> erstmal weiter "Walking" abspielen.

# PlayerLocation erweitern

## Variablen

```
[SerializeField] float movementThreshold = 0.1f;  
private Vector3 lastAcceleration;
```

## Funktion

```
void UpdateAcceleration()  
{  
    Vector3 current = Input.acceleration;  
    float delta = (current - lastAcceleration).magnitude;  
    lastAcceleration = current;  
  
    if (delta > _movementThreshold)  
        _playerAvatar.SetPlayerState(PlayerAvatar.PlayerState.Walking);  
    else  
        _playerAvatar.SetPlayerState(PlayerAvatar.PlayerState.Idle);  
}
```

Dann in **Update() (!)** aufrufen: `UpdateAcceleration();`

# Einmal im Live-Build testen!

Builden und Testen

Kurze Pause?

# Creatures Spawned

# Creature Spawner

## Creatures Spawnen

Wann, wo, wieviele?

- Random nach Spielstart
- Im Umkreis des Spielers -> z.B. 100m
- min. 3, maximal 5
- Despawnen wenn Distanz größer als 300m

Später kann man zusätzlich Map Data nutzen!

- In Flussnähe -> mehr "Wassermöner"
- In Parks -> mehr "Naturmonster"



# World-Space vs Map-Space

## World-Space bei Unity

- X / Y / Z
- “Meter-like”
- relative Positionen

## Map-/ Geo-Space: Reale Welt

- Lat / Long / Alt
- Bogen-Grad statt Meter

Müssen die Werte immer umrechnen!

# Creature Positionen

Die Creatures sollen auf der Map statisch sein!

Der Spieler steht still, die Map bewegt sich:

- müssen die Position der Creatures an die Geo-Koordinaten binden
- bei jeder Bewegung der Map, die Position in Unity neu berechnen
  - Dafür gibts den **CesiumGlobeAnchor**

# Creature Objekt

1. Neues GameObject / Prefab
  - a. **CesiumGlobeAnchor** Komponente hinzufügen
  - b. Neues Script erstellen und hinzufügen “**MapCreature**”
2. Im **MapCreature** Script
  - a. **CesiumGlobeAnchor** referenzieren
  - b. Spawn Funktion schreiben

```
public void SpawnCreature(Vector2 coordinates)
{
    _cesiumGlobeAnchor.longitudeLatitudeHeight = new double3(coordinates.y, coordinates.x, 0);
    _cesiumGlobeAnchor.Sync();

    AdjustHeight();
}
```

- c. **SpawnCreature()** zum testen in Start aufrufen (`SpawnCreature(new Vector2(8.65150f, 49.41439f))`);

# Creature Objekt testen

Creature Objekt Prefabisieren und aus Szene löschen.

1. Play drücken
2. Prefab in Szene ziehen. Als **Child von CesiumGeoreference!**
3. Bewegen. Die Creature sollte auf der Karte am selben Punkt bleiben
4. Sehr gut. Wir haben Spawnbare Creatures!

# Spawn Logik

Brauchen ein Spawner-Objekt.

Neues GameObject + Neues Script **CreatureSpawner**

1. Zufällige Position im Umkreis des Spielers finden
2. MapCreature an Zufallsposition spawnen
3. Müssen verzögert Spawnen! Erst wenn die Map geladen ist!
  - a. Am besten per Event
  - b. **jetzt reicht erstmal fest 3s**

# CreatureSpawner Script

## Variablen

```
float _metersPerDegreeLong;
```

```
const float _metersPerDegreeLat = 111320f; //
```

```
111.320m = 111,32km
```

```
[SerializeField] PlayerLocationForMap
```

```
_playerLocation;
```

```
[SerializeField] GameObject _creaturePrefab;
```

```
[SerializeField] float _radius;
```

```
[SerializeField] Transform _parentTransform;
```

```
[SerializeField] float _numberOfSpawns = 5;
```

# CreatureSpawner Script

## Funktionen

```
Vector2 GetRandomPointAroundPlayer()
{
    // Zufälliger Punkt im Kreis
    Vector2 random = Random.insideUnitCircle * _radius;

    // Umberechnung der MeterProGrad Longitude, alternativ auch einfach 111320f nehmen
    _metersPerDegreeLong = 111320f
        * Mathf.Cos(_playerLocation.GetPlayerCoords().x
        * Mathf.Deg2Rad);

    // Umrechnung Meter → GPS °
    var lat = _playerLocation.GetPlayerCoords().x + (random.y / _metersPerDegreeLat);
    var lon = _playerLocation.GetPlayerCoords().y + (random.x / _metersPerDegreeLong);

    return new Vector2(lat, lon);
}
```

# CreatureSpawner Script

Aufrufen mit Verzögerung

In Start()

```
Invoke("InitialSpawns", 3f);
```

```
void InitialSpawns()
{
    for (int i = 0; i < _numberOfSpawns; i++)
    {
        MapCreature newCreature = Instantiate(_creaturePrefab, _parentTransform).GetComponent<MapCreature>();
        newCreature.SpawnCreature(GetRandomPointAroundPlayer());
    }
}
```



Testen &  
Kurze Pause?

# Creature Encounter

# Creature Encounter

Es spawnen Creatures, aber wir können noch nichts mit ihnen machen!

Encounter starten, wenn:

- Der Spieler nah genug ist
  - Irgendwie sichtbar machen, dass der Spieler nah genug ist.
  - z.B. durch Partikel / Effekte / Animation
- Der Spieler auf die Creature tippt

# Creature Encounter - Entfernungsberechnung

Distanzen <200m! Es reicht aus, wenn wir Unity Distanz nehmen!

```
float dist = Vector3.Distance(playerPos, creaturePos);
```

Wollen aber nur die 2D Distanz (Höhenunterschied ignorieren)

Also vereinfacht (in **MapCreature**):

```
void CalculateDistanceToPlayer() {  
    Vector2 creature = new Vector2(this.transform.position.x, this.transform.position.z);  
    float dist = creature.magnitude;  
}
```

In Update() aufrufen

# Creature Encounter - Aktion beim nähern

## 1. CalculateDistanceToPlayer() erweitern

```
if (dist <= _minimumDistanceForCatching)
    SetCatchingVisualsActive(true);
else
    SetCatchingVisualsActive(false);
```

## 2. CatchingVisuals erstellen und in MapCreature Prefab anlegen (z.B. Particle System)

## 3. In SetCatchingVisuals verlinken

```
void SetCatchingVisualsActive(bool active) {
    _catchingVisuals.SetActive(active);}
```

# Creature Encounter - Aktion beim klicken

1. Dem **MapCreature** GameObject einen **Collider** hinzufügen
2. Die MapCreature Klasse erweitern um **IPointerClickHandler**

```
public class MapCreature : MonoBehaviour, IPointerClickHandler
```

3. **OnPointerClick** Funktion hinzufügen

```
public void OnPointerClick(PointerEventData eventData) {}
```

4. Dort Aktion definieren, z.B. SceneSwitch und an CatchingRange knüpfen

```
if (!_inCatchingRange)
```

```
return;
```

```
SceneManager.LoadScene("CreatureCatch");
```

5. Auf der **MainCamera** die Komponente **PhysicsRaycaster** hinzufügen

Testen &  
Kurze Pause?

# Verbessern der Szene

1. Animierte Creatures
2. Drehbare/Schwenkbare Kamera
3. Zoom
4. Partikeleffekte
5. Creatures Despawnen wenn zu weit weg



# Bonus Tasks

- Umstellen der Weitergabe der Locations auf Events
  - statt von PlayerLocation Aufrufe zu starten: PlayerLocation Invoked Event
  - alle Scripte die von PlayerLocation Updates brauchen fügen **.AddListener** auf das Event hinzu
  - Dadurch wird das PlayerLocation zu einem "Service" und braucht keine direkte Verknüpfungen mehr
- Auslagern der MockLocation-Daten in ein eigenes Script
  - wenn UseMockData = true, dann bei eigener Klasse Mock Daten anfordern
  - dadurch ist das Location Script sauberer
- Verzögern der Raycasts um ein Frame
  - es kann passieren, dass während des Position Updates ein Tile neu geladen wird.
  - die Raycast für AdjustHeight passieren aber noch im alten "Tick" und treffen das alte Tile.
  - Bei sehr steilen Terrains steckt der Spieler dadurch kurz im Boden